

RateIt Sprint 1 — GitHub Issues

Copy-paste issue set for repo handoff

Purpose: This file packages the Sprint 1 issue set into a single handoff document so a developer or AI coding team can drop the work straight into GitHub issues and start building.

Issue #1 — Monorepo scaffold and shared types

Labels: sprint-1, P0, foundation, types

Summary: Create the initial RateIt monorepo/app structure and shared TypeScript types used across the extension, admin web app, and API.

Goal: Set up a clean project foundation so all apps share the same core models and enums.

Scope

- Create apps/extension, apps/admin-web, apps/api, packages/shared-types, packages/verdict-engine, and optional packages/ui.
- Define shared TypeScript types for Verdict, PillarRating, Merchant, Assessment, and UserReport.

Requirements

- Use TypeScript throughout.
- Shared types must import cleanly across apps.
- Add minimal local setup instructions.
- Keep scaffold clean and minimal.
- No unnecessary product logic in this issue.

Acceptance Criteria

- Repo installs and runs locally.
- Shared types compile and import correctly.
- Monorepo structure is clean and documented.
- README includes install/run steps.
- No dead placeholder code.

Notes

- Verdict = Trusted | Caution | High Risk | Insufficient Data
- PillarRating = Strong | Mixed | Weak | Unknown

Issue #2 — Database schema and seed data

Labels: sprint-1, P0, backend, database

Summary: Create the initial database schema for RateIt and seed it with four test merchants covering all verdict states.

Goal: Provide a working data layer for merchants, assessments, user reports, and admin users.

Scope

- Create entities/tables for Merchant, Assessment, UserReport, and AdminUser.
- Seed one merchant each for Trusted, Caution, High Risk, and Insufficient Data.

Requirements

- Merchant minimum fields: id, name, domain, category, currentVerdict, lastReviewedAt, publicSummary, status.
- Assessment minimum fields: merchantId, transparencyRating, reliabilityRating, integrityRating, communicationRating, redFlags, internalRationale, publicReasons, reviewedBy, reviewedAt.
- UserReport minimum fields: merchantId or domain, reportType, description, optionalScreenshotUrl, status, createdAt.

Acceptance Criteria

- Database can be created from scratch.
- Migration works.
- Seed script works.
- Four sample merchants appear in DB.
- Seeded records are visible to admin/API later.

Issue #3 — Verdict engine

Labels: sprint-1, P0, logic, rules

Summary: Build the RateIt verdict engine as pure TypeScript functions that convert pillar ratings and red flags into a final verdict.

Goal: Create a deterministic, testable rules engine shared by admin and API.

Requirements

- Inputs: transparencyRating, reliabilityRating, integrityRating, communicationRating, redFlags[], and optional confidence/evidence metadata.
- Outputs: Trusted, Caution, High Risk, Insufficient Data.
- Rules: Trusted = at least 3 strong pillars and no major red flags; Caution = mixed profile, one weak pillar or multiple medium concerns; High Risk = 2+ weak pillars, or severe red-flag

cluster, or strong harmful pattern; Insufficient Data = not enough meaningful evidence / unknown profile.

- Pure TypeScript functions only; no UI in this issue.
- Avoid over-engineering.

Acceptance Criteria

- Verdict function is deterministic.
- Main paths are covered by tests.
- Edge cases are covered by tests.
- Logic is readable and easy to maintain.

Notes

- Tests required: clear Trusted, clear Caution, clear High Risk, clear Insufficient Data, and one conflicting-input edge case.

Issue #4 — Admin shell and merchant list

Labels: sprint-1, P1, admin, frontend

Summary: Create the first admin screens: login shell, merchant list page, and merchant detail/edit page shell.

Goal: Allow admin users to see seeded merchants and navigate into merchant records.

Scope

- Build basic admin shell/layout, login shell, merchant list page, merchant detail/edit page shell, and a Create Merchant action.

Requirements

- Merchant list columns: name, domain, verdict, last reviewed date, status.
- Keep styling minimal and clean.
- Focus on routing and functionality.
- Authentication can be basic/mock for sprint 1.

Acceptance Criteria

- Admin can open the app.
- Admin can see seeded merchants.
- Admin can click into a merchant record.
- Create Merchant action exists.
- Basic routing works end-to-end.

Issue #5 — Merchant assessment form

Labels: sprint-1, P0, admin, form

Summary: Build the internal merchant assessment form used to create and update merchant trust records.

Goal: Allow admin users to enter trust data and trigger verdict recalculation.

Requirements

- Fields: store name, domain, category, status, last reviewed date, transparency rating, reliability rating, integrity rating, communication rating, red flags, internal rationale, public summary, top public reasons (3–4 items).
- Persist form data to database.
- Use shared verdict engine on save.
- Show current verdict on form.
- Basic required-field validation.
- UX should be review-oriented, not flashy.

Acceptance Criteria

- Admin can create a merchant assessment.
- Admin can update an existing assessment.
- Saved values persist correctly.
- Final verdict recalculates on save.
- Current verdict is visible in form.
- Validation prevents empty required saves.

Issue #6 — API endpoint for merchant lookup by domain

Labels: sprint-1, P0, api, backend

Summary: Create the API endpoint used by the Chrome extension to retrieve merchant trust data by domain.

Goal: Allow the extension to look up a store and display a verdict with reasons.

Requirements

- Endpoint: GET /merchant-by-domain?domain=<domain>
- Known merchant response must return domain, merchant name, verdict, lastReviewedAt, pillar ratings snapshot, publicSummary, and topReasons.
- Unknown merchant response must return a valid Insufficient Data payload with a stable response shape.
- Use shared types.

- Add basic error handling.

Acceptance Criteria

- Seeded known merchants resolve correctly.
- Unknown domain returns Insufficient Data payload.
- Response shape is typed and stable.
- Basic errors handled cleanly.

Notes

- Tests required: known domain, unknown domain, malformed request.

Issue #7 — Admin CRUD API for merchants and assessments

Labels: sprint-1, P1, api, admin

Summary: Build backend routes for admin create/update/list/get merchant functionality.

Goal: Support the admin UI with working CRUD endpoints and verdict recalculation.

Requirements

- Endpoints: create merchant, update merchant, get merchant by id, list merchants.
- Validate required fields.
- Use shared verdict engine on create/update.
- Return typed payloads.
- Keep auth minimal/simple for sprint 1.

Acceptance Criteria

- Admin form can save merchant records end-to-end.
- Admin can retrieve merchant by id.
- Merchant list endpoint works.
- Verdict recalculates on save/update.
- Invalid data is rejected cleanly.

Issue #8 — Chrome extension popup MVP

Labels: sprint-1, P0, extension, frontend

Summary: Build the RateIt Chrome extension popup that detects the current domain, calls the backend, and displays the trust verdict.

Goal: Deliver the first user-facing RateIt experience.

Requirements

- Popup must show: RateIt Signal header, current domain, verdict, one-line explanation, last reviewed date, four-pillar snapshot, and top reasons.
- Buttons in UI: View Full Report, Report This Store, Why This Rating?
- Detect current browser tab domain.
- Call GET /merchant-by-domain.
- Support all four verdict states.
- Code should be modular and clean.

Acceptance Criteria

- Extension detects current domain.
- Known merchant verdict renders correctly.
- Unknown merchant renders Insufficient Data.
- Popup shows required sections.
- Buttons are present in UI.

Issue #9 — Report This Store flow

Labels: sprint-1, P2, reports, frontend, backend

Summary: Build a minimal user reporting flow so shoppers can submit a store report.

Goal: Capture useful user-submitted reports and store them for admin review.

Requirements

- Form fields: domain/store, report type, short description, optional screenshot URL placeholder field.
- Submit report to backend.
- Save as UserReport.
- Show success/error state.
- Keep flow simple and functional.

Acceptance Criteria

- User can submit a report.
- Report is saved in database.
- Basic success/error state shown.
- Admin can view reports later via backend data.

Issue #10 — End-to-end smoke test for Sprint 1 walking skeleton

Labels: sprint-1, P0, testing, integration

Summary: Verify the full Sprint 1 walking skeleton works from admin entry to extension display.

Goal: Confirm the product loop functions end-to-end before calling Sprint 1 complete.

Requirements

- Required scenarios: known Trusted domain, known Caution domain, known High Risk domain, unknown domain, and admin updates merchant assessment with extension reflecting the change.
- Add automated coverage where practical.
- Document manual test steps if needed.
- Verify no broken contract between admin, API, DB, and extension.

Acceptance Criteria

- All required scenarios pass.
- Setup/test steps are documented.
- Admin update is reflected in extension output.
- No broken response contracts between apps.